

CSCE 735 PARALLEL COMPUTING · MODULE 02 · LECTURE NOTES

Parallel Architectures

The machines you are writing for.

MODULE OUTCOMES

Distinguish the forms of parallelism available in hardware.

Explain SIMD and what makes a loop vectorisable.

Explain NUMA and its consequences for data placement.

Contrast latency-oriented and throughput-oriented cores.

Choose a programming model from the hardware and the problem.

1 Forms of parallelism

Form	Granularity	Who exploits it	Covered in
Instruction-level (ILP)	Independent instructions within one stream.	The hardware, automatically — out-of-order execution, superscalar issue, speculation (CSCE 614).	—
SIMD / vector	Elements processed by a single instruction.	The compiler, if you let it — or you, via intrinsics.	This module
Thread and core	Independent instruction streams sharing memory.	You, explicitly.	Modules 03–07
Node	Separate machines with separate memory.	You, via explicit messages.	Module 10

INSTRUCTION-LEVEL PARALLELISM IS ALREADY SPENT

Out-of-order execution, speculative execution, and superscalar issue extract parallelism from a sequential instruction stream automatically, with no cooperation from the programmer. This is the machinery CSCE 614 covered, and it is why a serial program runs far faster than its instruction count would suggest.

And it has reached diminishing returns. The amount of parallelism extractable is bounded by how far ahead the processor can see, which is bounded by branch prediction accuracy — and by the quadratic cost of the scheduling hardware as the window grows.

So the hardware cannot find much more on its own. Wider issue and deeper windows have stopped paying.

Which is precisely why the remaining forms require you. The parallelism that could be extracted automatically has been extracted; what remains must be *expressed* in the structure of the program.

2 SIMD

ONE INSTRUCTION, SEVERAL ELEMENTS

A 512-bit vector register holds sixteen single-precision floats, or eight doubles, or sixty-four bytes. **One vector add instruction adds all sixteen pairs.**

So a vectorised loop can run four to sixteen times faster than the equivalent scalar loop, at identical clock speed and on a single core.

It is free in the sense that the hardware is already present — you paid for the vector units when you bought the processor, and they sit idle if the code does not use them.

And they are idle by default. Scalar code occupies one lane of sixteen, which is a 94% waste of the vector unit on every arithmetic instruction. **This is the cheapest available performance and the most commonly left unclaimed.**

```
// VECTORISES: independent, contiguous, no branches
for (int i = 0; i < n; i++) c[i] = a[i] + b[i];

// NO: loop-carried dependence
for (int i = 1; i < n; i++) a[i] = a[i-1] + b[i];

// NO (usually): compiler cannot prove a and b do not overlap
void f(float* a, float* b, int n) { for (...) a[i] = b[i] * 2; }
// fix: float* __restrict__ a

// POORLY: non-contiguous needs gather/scatter
for (int i = 0; i < n; i++) c[i] = a[idx[i]];

// POORLY: divergent branch -- both sides execute, masked
for (int i = 0; i < n; i++) if (a[i] > 0) c[i] = f(a[i]);
```

Requirement	Why	Fix when violated
Independent iterations	Element i cannot depend on element $i-1$, since the vector computes them simultaneously.	Restructure the algorithm — a scan (Module 11) is the standard answer for prefix dependencies.
Contiguous access	A vector load reads consecutive memory. Scattered access requires gather instructions, which are far slower.	Structure of arrays rather than array of structures.
No aliasing	If two pointers might refer to overlapping memory, the compiler must assume a write through one could affect a read through the other.	<code>__restrict__</code> , which promises they do not overlap. The most common blocker and the easiest fix.
Uniform control flow	All lanes execute the same instruction. A branch that goes different ways in different lanes means both paths execute with masking.	Make the branch uniform, or accept the cost, or use a branchless formulation.

- **Read the compiler's vectorisation report.** `-fopt-info-vec` and `-fopt-info-vec-missed` on GCC, `-Rpass-analysis=loop-vectorize` on Clang. **It tells you what it vectorised and, far more usefully, why it declined** — and the reasons are specific and actionable.
- **Use `__restrict__` on pointer parameters** you know do not alias. This alone unlocks a great deal of vectorisation.
- **Structure of arrays, not array of structures.** An array of particle structs gives strided access when a loop touches one field; separate arrays per field give contiguous access. **This is frequently a four- to eightfold difference and it is a data layout decision rather than an optimisation** — which means it is expensive to change later.
- **Align data to the vector width** and tell the compiler it is aligned. Unaligned loads are supported and slower.
- **Intrinsics as a last resort.** They are unportable across instruction sets, unreadable, and occasionally the only way to express what you need. Write the portable version first and keep it.

3 Memory

NUMA: NOT ALL MEMORY IS EQUALLY FAR AWAY

On a multi-socket machine, each socket has its own memory controller and its own attached memory. A core accessing memory attached to its own socket goes directly; a core accessing the other socket's memory crosses an interconnect.

Remote access costs roughly 1.5 to 2 times the latency and offers lower bandwidth. On a four-socket machine the disparity is larger.

And allocation is first-touch. malloc reserves address space; the physical page is assigned when it is first written, **on the socket of the thread that writes it** — not the one that allocated it.

So initialise your data in parallel, using the same thread-to-data mapping you will use when computing. A serial initialisation loop places every page on one socket, and every thread on the other socket then performs all its accesses remotely. **This can halve your effective bandwidth**, and it is invisible in the code — the initialisation loop looks harmless.

FALSE SHARING DESTROYS PERFORMANCE WITHOUT AFFECTING CORRECTNESS

Cache coherence operates on cache lines, typically 64 bytes, not on individual variables. Two threads writing to *different* variables that happen to occupy the same cache line will invalidate each other's copy of that line on every single write.

The program is entirely correct. There is no data race, no undefined behaviour, and the answer is right. Thread sanitisers report nothing.

And it can be slower than the serial version, because every write turns into a coherence transaction across the interconnect instead of hitting in local cache. A per-thread counter array is the classic case: eight threads incrementing counts[tid] in an array of eight integers all share one cache line.

The fix is padding — give each thread's data its own cache line, by padding the struct or over-allocating the array. **The general pattern is per-thread accumulators combined at the end**, which avoids both false sharing and contention, and is the right structure for almost all reductions anyway (Module 06).

4 Latency machines and throughput machines

	CPU — latency oriented	GPU — throughput oriented
Cores	Few and complex — tens.	Many and simple — thousands of lanes.
Per-core machinery	Large caches, deep out-of-order windows, aggressive branch prediction, prefetchers.	Small caches, in-order execution, no speculation.
Goal	Finish one thread as quickly as possible.	Maximise total work completed per second , with no concern for any individual thread's latency.

	CPU — latency oriented	GPU — throughput oriented
Latency hiding	Caches, prefetching, and speculation — predict and avoid the stall.	Switch to another thread. With thousands resident, there is always one ready, so the stall is covered by other work rather than avoided.
Good at	Branches, irregular access, pointer chasing, serial sections, low-latency response.	Regular, data-parallel, arithmetic-heavy work with predictable access patterns.

The difference in latency-hiding mechanism is the key one, and it explains essentially everything about GPU programming in Modules 08 and 09: why occupancy matters, why divergence is expensive, and why a GPU kernel with too few threads performs badly even when the arithmetic is trivial.

Problem characteristics	Appropriate model
Regular, data-parallel, arithmetic-heavy, predictable access.	GPU (Modules 08, 09).
Irregular, branchy, pointer-chasing, dependent on data values.	CPU threads (Modules 03–06).
Embarrassingly parallel — independent tasks with no shared state.	Threads or processes; almost anything works.
Data larger than one machine's memory.	Distributed memory, MPI (Module 10). This is frequently the actual reason for distribution rather than speed.
Small, latency-critical work.	One thread. Thread creation costs microseconds and synchronisation costs more; for small work, parallelism costs more than it saves , and the correct decision is not to parallelise.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Stanford CS149 — architecture lectures (free)	The forms of parallelism, SIMD, and the latency/throughput distinction. The best free treatment of §4.
Agner Fog — optimisation manuals (free)	Instruction tables, microarchitecture details, and vectorisation guidance. The reference for §2, and exhaustively measured.
Ulrich Drepper — What Every Programmer Should Know About Memory (free)	NUMA, cache coherence, and false sharing (§3), in far more detail than anywhere else. Long, and worth it.
Intel — Intrinsics Guide (free)	The reference when you reach the last resort in §2.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a vectorisable loop and compile with vectorisation reporting. Read the output.
- 2 Introduce each of the four blockers in §2 in turn and record what the compiler says about each.
- 3 Fix the aliasing case with `__restrict__` and measure the speedup.
- 4 Implement the same particle update with array-of-structures and structure-of-arrays layouts. Measure both and report the ratio.
- 5 **Demonstrate false sharing:** have eight threads increment adjacent elements of an array, then pad to separate cache lines and measure again.
- 6 On a multi-socket machine, initialise an array serially and then in parallel, and measure the bandwidth of a subsequent parallel loop over it.
- 7 Measure memory bandwidth with a STREAM-style benchmark and compare against the hardware's specification.
- 8 Time a trivial parallel region and find the problem size below which parallelisation loses.
- 9 Profile a program and determine what fraction of its arithmetic instructions are vector instructions.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name four forms of hardware parallelism and who exploits each.
- 2 Why has instruction-level parallelism reached diminishing returns?
- 3 How much work does one 512-bit vector instruction do, and what does scalar code waste?
- 4 Give four requirements for a loop to vectorise and the fix for each when violated.
- 5 Why does structure-of-arrays beat array-of-structures?
- 6 What is first-touch allocation and what must you do about it?
- 7 What is false sharing, why is the program still correct, and what is the fix?
- 8 Contrast CPU and GPU on cores, latency hiding, and what each is good at.
- 9 Give a case where the right decision is not to parallelise.